Experiment No:15
Experiment to understand the data preprocessing in Data Science

AIM:
To understand the data preprocessing in Datascience

DESCRIPTION:
Understand the importance of Data preprocessing in data science

CODE:

```
import numpy as np
import pandas as pd

# Create sample dataset
df = pd.DataFrame({
    'Country': ['France', 'Spain', np.nan, 'Spain', 'Germany',
                'France', 'Spain', 'Germany', 'France',
'Germany'],
    'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],
    'Salary': [72000, 48000, 54000, 61000, np.nan,
               58000, 52000, 79000, 83000, 67000],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes',
                  'Yes', 'No', 'Yes', 'No', 'Yes']
})

# Save dataset
df.to_csv('/content/preprocessing_sample.csv', index=False)

# Read dataset
df = pd.read_csv('/content/preprocessing_sample.csv')
print(df)
print("\n\n")
```

```
# Display dataset information
df.info()
print("\n\n")

# Display Country column
print(df['Country'])
print("\n\n")

# Find mode of Country
print(df['Country'].mode())
print("\n\n")

# Display first mode
print(df['Country'].mode()[0])
print("\n\n")

# Display type of mode
print(type(df['Country'].mode()))
print("\n\n")

# Fill missing values
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(round(df['Salary'].mean()))

# Display updated dataset
print(df)
print("\n\n")

# Convert Country into dummy variables
print(pd.get_dummies(df['Country']))
print("\n\n")
```

```
# Combine dummy variables with other columns
updated_dataset = pd.concat(
    [pd.get_dummies(df['Country']), df.iloc[:, [1, 2, 3]]],
    axis=1
)

print(updated_dataset)
print("\n\n")

# Display dataset information
updated_dataset.info()
print("\n\n")

# Convert Purchased values into numbers
updated_dataset['Purchased'] =
updated_dataset['Purchased'].replace(
    ['No', 'Yes'], [0, 1]
)

# Display final dataset
print(updated_dataset)
```

OUTPUT:

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	NaN	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	Germany	48.0	79000.0	Yes
8	France	50.0	83000.0	No
9	Germany	37.0	67000.0	Yes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Country    9 non-null      object 
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object 
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
0     France
1      Spain
2        NaN
3      Spain
4    Germany
5     France
6      Spain
7    Germany
8     France
9    Germany
Name: Country, dtype: object
```

```
0     France
1    Germany
2      Spain
Name: Country, dtype: object
```

France

<class 'pandas.core.series.Series'>

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	France	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	63778.0	Yes
5	France	35.0	58000.0	Yes
6	Spain	38.0	52000.0	No
7	Germany	48.0	79000.0	Yes
8	France	50.0	83000.0	No
9	Germany	37.0	67000.0	Yes

	France	Germany	Spain
0	True	False	False
1	False	False	True
2	True	False	False
3	False	False	True
4	False	True	False
5	True	False	False
6	False	False	True
7	False	True	False
8	True	False	False
9	False	True	False

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	No
1	False	False	True	27.0	48000.0	Yes
2	True	False	False	30.0	54000.0	No
3	False	False	True	38.0	61000.0	No
4	False	True	False	40.0	63778.0	Yes
5	True	False	False	35.0	58000.0	Yes
6	False	False	True	38.0	52000.0	No
7	False	True	False	48.0	79000.0	Yes
8	True	False	False	50.0	83000.0	No
9	False	True	False	37.0	67000.0	Yes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   France     10 non-null     bool
 1   Germany    10 non-null     bool
 2   Spain      10 non-null     bool
 3   Age        10 non-null     float64
 4   Salary     10 non-null     float64
 5   Purchased  10 non-null     object
dtypes: bool(3), float64(2), object(1)
memory usage: 402.0+ bytes
```

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	0
1	False	False	True	27.0	48000.0	1
2	True	False	False	30.0	54000.0	0
3	False	False	True	38.0	61000.0	0
4	False	True	False	40.0	63778.0	1
5	True	False	False	35.0	58000.0	1

```
6   False    False   True  38.0  52000.0          0
7   False     True  False  48.0  79000.0          1
8    True    False  False  50.0  83000.0          0
9   False     True  False  37.0  67000.0          1
```

```
/tmp/ipykernel_1895/1211043690.py:70: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  updated_dataset['Purchased'] = updated_dataset['Purchased'].replace(
```

RESULT:

Thus, data preprocessing was successfully performed using Pandas and NumPy. Missing values were handled using mode, median, and mean imputation. Categorical data in the `Country` column was converted into dummy variables, and the `Purchased` column was encoded into numerical values (0 and 1). The final dataset was cleaned and prepared for further data analysis and machine learning.